

Cloud Security with AWS IAM

AWS Cloud Portfolio Project

Roman Pillay

BSc Information Technology (Final Year)

romanpillay72@gmail.com

AWS IAM

Amazon EC2

Cloud Security

JSON Policies

Access Control

Introducing Today's Project!

Project overview

In this project, I will demonstrate how to use AWS IAM to control access and permissions settings in my AWS account. I'm doing this project to learn about cloud security from the absolute foundations - every company thinks about access permissions, and there are even entire jobs focused on the skills i'm about to build today.

Tools and concepts

Services I used were Amazon Ec2 and AWS IAM. Key concepts I learnt include IAM users, policies, user groups and account aliases. I also learnt how to use policy simulator and how JSON policies work. AS well as how to launch an instance, how to tag an instance, how to log in as another user.

Project reflection

This project took me approximately 1.5 hours including demo time. The most challenging part was understanding the IAM policy since it was written in JSON and it contained multiple statements. It was most rewarding to see permission denied when the intern tried to delete the production instance, which meant the IAM access management setup was working.

Tags

What I did in this step

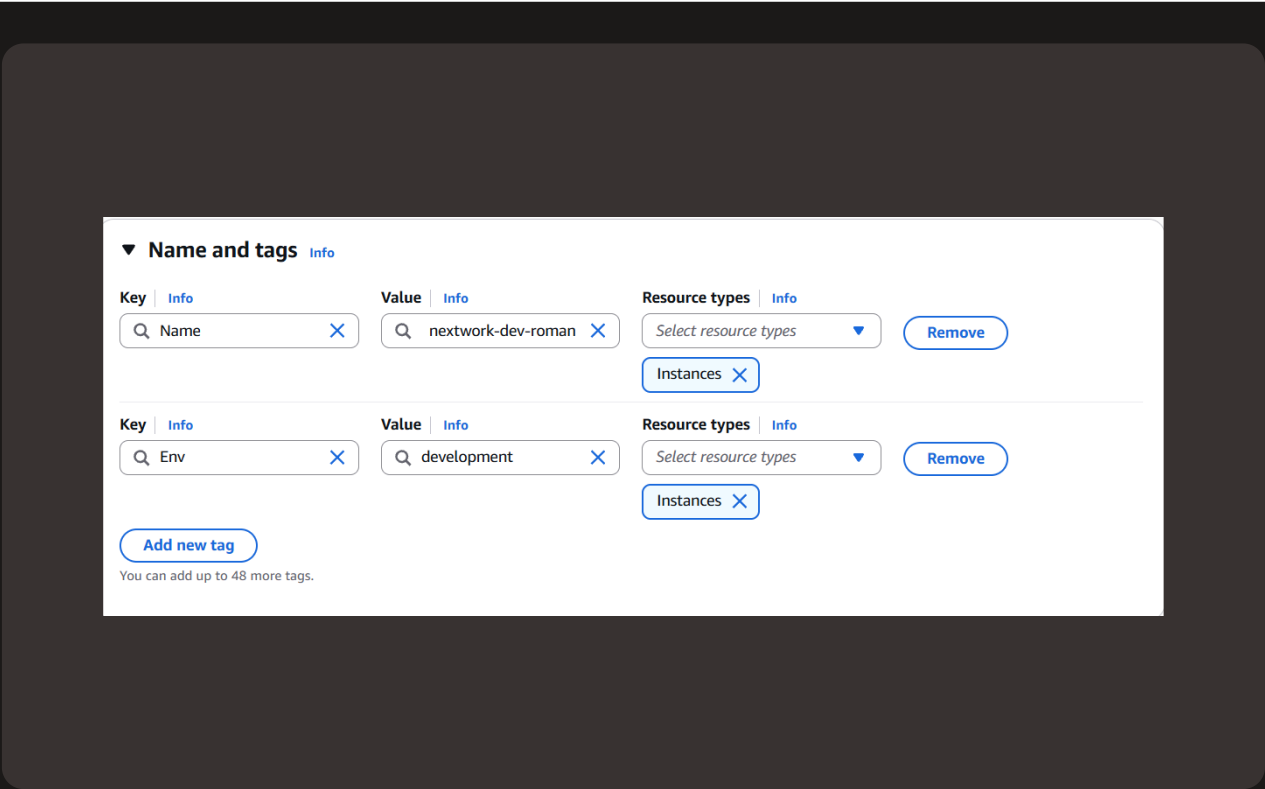
In this step, I will launch two EC2 instances because we need to boost computing power, we're expecting more users and traffic into the website.

Understanding tags

Tags are organisational tools that lets me label my resources. They helpful for grouping resources, cost allocation and applying policies for all respurces with the same tag.

My tag configuration

The tag I've used on my EC2 instances is called Env, which stands for environment. The value I've assigned for my instances are production and development.



IAM Policies

What I did in this step

In this step, I will use IAM policies to control the access level of a new Nextwork intern because they should have access to the development environment(i.e the development instance) and NOT the production environment.

Understanding IAM policies

IAM Policies are like rules that determine who can do what in the AWS account. I'm using policies today to control who has access to the production/environment instance.

The policy I set up

For this project, I've set up a policy using JSON.

Policy effect

I've created a policy that allows the policy holder to have permission to do anything they want to any instance tagged with "development". They can also see information for any instance, but they're denied access to deleting or creating tags for any instance as well.

Understanding Effect, Action, and Resource

The Effect, Action, and Resource attributes of a JSON policy means whether or not the policy is allowing or denying action(i.e Effect); what the policy holder can or cannot do (i.e Action); and the specific AWS resources that the policy relates to (i.e resource).

My JSON Policy

Policy editor

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "ec2:*",  
7       "Resource": "*",  
8       "Condition": {  
9         "StringEquals": {  
10          "ec2:ResourceTag/Env": "development"  
11        }  
12      }  
13    },  
14    {  
15      "Effect": "Allow",  
16      "Action": "ec2:Describe*",  
17      "Resource": "*"   
18    },  
19    {  
20      "Effect": "Deny",  
21      "Action": [  
22        "ec2:DeleteTags",  
23        "ec2:CreateTags"  
24      ],  
25      "Resource": "*"   
26    }  
27  ]  
28 }  
29
```

Account Alias

What I did in this step

In this step, I will set up an account alias which is like a nickname for our AWS account's console login. This is because an account alias makes it simpler for our users to login

Understanding account aliases

An account alias is simply a nickname for our AWS account instead of a long account ID, I can now reference my account alias instead.

Setting up my account alias

Creating an account alias took me 30 seconds, it's a simple configuration in the IAM dashboard. Now, my new AWS console sign-in URL uses the alias instead of my account ID.

Create alias for AWS account 991416557248

Preferred alias

nextwork-alias-roman

Must be not more than 63 characters. Valid characters are a-z, 0-9, and - (hyphen).

New sign-in URL

<https://nextwork-alias-roman.signin.aws.amazon.com/console>

 IAM users will still be able to use the default URL containing the AWS account ID.

Cancel

Create alias

IAM Users and User Groups

What I did in this step

In this step, I will set up two resources IAM users and IAM user groups. This is because IAM users are like logins for people that want access to our AWS account, while user groups are like folders to manage users that have the same level of access

Understanding user groups

IAM user groups are like folders that collect IAM users so that you can apply permission settings at the group level.

Attaching policies to user groups

I attached the policy I created to this user group, which means any means user created inside this group will automatically get the permissions attached to the policy.

Understanding IAM users

IAM users are people or entities that have access or can login to your AWS account.

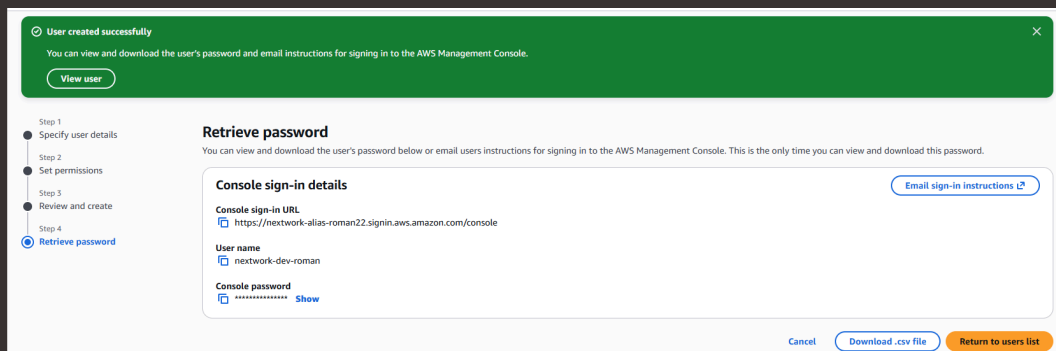
Logging in as an IAM User

Sharing sign-in details

The first way is to email sign in instructions to the user, while the second way is to download a .csv file with the sign in details inside.

Observations from the IAM user dashboard

Once I logged in as my IAM user, I noticed that the user is already denied access to panels on the main AWS dashboard. This was because i only set up permissions to the EC2 instances, so the intern would not have access to even see anything else.



Testing IAM Policies

What I did in this step

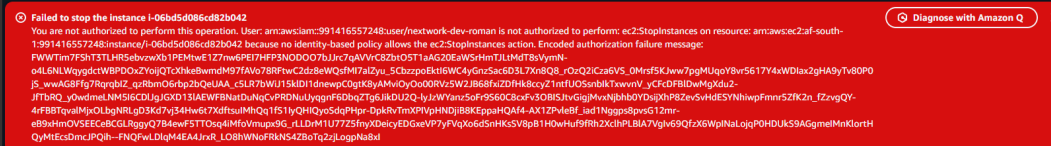
In this step, I will login to my own AWS account as the intern and test access to the production and development instances because i want to make sure that the intern does not have the ability to do anything that can affect the production environment.

Testing policy actions

I tested my JSON IAM policy by attempting to stop both the development and production instances.

Stopping the production instance

When I tried to stop the production instance i was met with an error. This was because the production instance is tagged with the production label which is outside of the scope of the permission policy, interns are only allowed to do things to the development instance.



Stopping the development instance

Next, when I tried to stop the development instance I successfully saw the instance state changed to stopping then it stopped. This was because the permission policy allows users in the group to stop instances.

A screenshot of an AWS console success message. It is a green bar with a white checkmark icon on the left, followed by the text "Successfully initiated stopping of i-0bfb5cca412a7ec0".

Successfully initiated stopping of i-0bfb5cca412a7ec0

IAM Policy Simulator

To extend my project, I'm going to test the permission policies in a safer and more controlled way, a tool called the IAM policy simulator. I'm doing this because having to stop instances and log into AWS accounts as other users is disruptive. So I wanted to find a more efficient way.

Understanding the IAM Policy Simulator

The IAM Policy Simulator is a tool that lets me simulate actions and test permission settings by defining a specific user/group/role and the action I want to test for. It's useful for saving time when testing permission settings. No more logging into another user or stopping resources.

How I used the simulator

I set up a simulation for whether the dev user group has permission to stop instances or delete tags. The results were denied for both so I had to adjust the scope for the EC2 instances to ones that are tagged with development. Once I applied that tag, permission was allowed.

Policy Simulator

Amazon EC2

2 Action(s) sele...

Select All

Deselect All

Reset Contexts

Clear Results

Run Simulation

Global Settings ⓘ

Action Settings and Results

[2 actions selected. 0 actions not simulated. 1 actions allowed. 1 actions denied..]

	Service	Action	Resource Type	Simulation Resource	Permission
▶	Amazon EC2	DeleteTags	not required	*	denied 1 matching statements.
▶	Amazon EC2	StopInstances	instance	*	allowed 1 matching statements.